



**JKG INTERNATIONAL
SCHOOL**

***INFORMATICS PRACTICES
(CODE-065)***

PROJECT REPORT ON HANGMAN

Session: 2025-2026

SUBMITTED BY:

SUBMITTED TO:

**NAME: S.U.PRANAV
SRIKANTH**

MS.SUNITA RAINA

CLASS: 12 A

ROLL No. -

CERTIFICATE

This is to certify that S.U.Pranav Srikanth, student of Class 12th, has successfully completed the Informatics Practices Project.

This is a bonafide piece of work carried out by the student as part of the CBSE curriculum, in partial fulfillment of the requirements for the Board Examination. The project has been completed under my supervision and guidance.

**SIGNATURE OF
INTERNAL EXAMINER**

**SIGNATURE OF
EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

First of all, I extend my heartiest gratitude to my teacher Mrs.Sunita Raina under whose utmost guidance this work could take the recent shape.

I wish to express my deep gratitude and sincere thanks to Principal Mrs. Nidhi Gaur, JKG International School, Indirapuram for the encouragement and for all facilities that she provided for this project work.

I can't forget to offer my sincere thanks to my classmates whose constant feedback makes my project more effective. Lastly, I would like to thank my parents for their encouragement in fulfilling my project.

S.U.Pranav Srikanth
12 A

INDEX

1	Introduction
2	Theoretical Background
3	Problem Definition
4	System Implementation
5	Source Code
6	Program Output
7	Conclusion
8	Bibliography

INTRODUCTION

The Hangman project is a Python-based game that brings the classic *Hangman* word-guessing challenge to life using a graphical user interface. Instead of playing in the console, this version uses Pygame to create an interactive and visually appealing experience.

In this game, the player must guess the hidden word one letter at a time. For each incorrect guess, a part of the hangman figure is drawn on the screen. The goal is to guess the correct word before the full figure is completed.

This project demonstrates the integration of Python programming, GUI design, and game logic, making it both entertaining and educational. It helps beginners understand how event-driven programming works and how to handle user input, display graphics, and manage game states effectively.

THEORETICAL BACKGROUND

The Hangman project is developed using Python and Pygame, which is a popular library for building games and interactive applications. Pygame provides easy-to-use functions for handling graphics, text, and user input, making it ideal for creating simple 2D games.

In this project, Pygame is used to draw the hangman figure, display letters and words, and respond to the player's guesses. The game window updates in real time whenever the player chooses a letter. Each wrong guess adds a new part to the hangman drawing, while correct guesses reveal letters of the hidden word.

The project combines Python's logical programming with Pygame's graphical features to make a visually interactive version of the traditional Hangman game. It helps players improve their vocabulary and spelling skills while providing an enjoyable gaming experience.

PROBLEM DEFINITION

Traditional Hangman games are usually played using text-based interfaces or on paper, which limits interactivity and user engagement. Players often find it difficult to visualize their progress, and the lack of graphical elements makes the gameplay less appealing.

To overcome these limitations, the Hangman GUI project aims to create a graphical version of the Hangman game using Python's Pygame library. This approach provides a more interactive and visually engaging experience, allowing users to clearly see the hangman figure, guessed letters, and remaining attempts.

The main problem addressed by this project is the lack of a simple, user-friendly, and visually interactive Hangman game that helps users enjoy and learn at the same time.

SYSTEM IMPLEMENTATION

SOFTWARE REQUIREMENT

OPERATING SYSTEM

Windows OS 10/11

PYTHON VERSION

Python IDLE version 3.12

DOCUMENTATION TOOL

Microsoft Word

HARDWARE REQUIREMENT

PROCESSOR

Intel Core i3 (minimum), Intel Core i5/i7 or higher
(recommended)

MEMORY (RAM)

2 GB - 4 GB

STORAGE

500 MB(minimum) or higher(recommended)

MONITOR

Standard VGA/LED monitor

KEYBOARD

Standard Keyboard

SOURCE CODE

main.py

```
import pygame
import random
import word

pygame.init()
WIDTH, HEIGHT = 600, 600
win = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Hangman")
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
RED = (220, 0, 0)
GREEN = (0, 170, 0)
GRAY = (160, 160, 160)
WORD_FONT = pygame.font.SysFont(None, 60)
MSG_FONT = pygame.font.SysFont(None, 40)
SMALL_FONT = pygame.font.SysFont(None, 28)
word_to_guess = random.choice(word.words).upper()
guessed = []
hangman_status = 0
max_wrong = 6
game_over = False
message = ""

def draw_hangman(win, status):
    t = 5
```

```

base_y = HEIGHT // 2 + 130
cx = WIDTH // 2
pygame.draw.line(win, BLACK, (cx - 100, base_y), (cx + 100,
base_y), t)
pygame.draw.line(win, BLACK, (cx - 80, base_y), (cx - 80,
base_y - 300), t)
pygame.draw.line(win, BLACK, (cx - 80, base_y - 300), (cx +
60, base_y - 300), t)
pygame.draw.line(win, BLACK, (cx + 60, base_y - 300), (cx +
60, base_y - 250), t)
if status > 0:
    pygame.draw.circle(win, BLACK, (cx + 60, base_y - 230),
20, t)
if status > 1:
    pygame.draw.line(win, BLACK, (cx + 60, base_y - 210),
(cx + 60, base_y - 130), t)
if status > 2:
    pygame.draw.line(win, BLACK, (cx + 60, base_y - 190),
(cx + 30, base_y - 160), t)
if status > 3:
    pygame.draw.line(win, BLACK, (cx + 60, base_y - 190),
(cx + 90, base_y - 160), t)
if status > 4:
    pygame.draw.line(win, BLACK, (cx + 60, base_y - 130),
(cx + 30, base_y - 90), t)
if status > 5:
    pygame.draw.line(win, BLACK, (cx + 60, base_y - 130),
(cx + 90, base_y - 90), t)

```

```

def display_word():
    disp = ""
    for l in word_to_guess:
        disp += l + " " if l in guessed else "_"
    text = WORD_FONT.render(disp.strip(), True, BLACK)
    rect = text.get_rect(center=(WIDTH // 2, HEIGHT - 120))
    win.blit(text, rect)

def draw_stats():
    correct = sum([1 for l in word_to_guess if l in guessed])
    wrong = hangman_status
    stats = SMALL_FONT.render(f"Correct: {correct} | Wrong: {wrong}", True, BLACK)
    win.blit(stats, (20, 20))

def draw_letters():
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    cols = 13
    start_x = 40
    start_y = HEIGHT - 80
    gap = 40
    for i, letter in enumerate(alphabet):
        color = GREEN if letter in guessed and letter in word_to_guess else RED if letter in guessed else BLACK
        text = SMALL_FONT.render(letter, True, color)
        row = i // cols
        col = i % cols
        x = start_x + col * gap
        y = start_y + row * 35

```

```
win.blit(text, (x, y))
```

```
def show_message(msg, color=BLACK, y_offset=0):  
    text = MSG_FONT.render(msg, True, color)  
    rect = text.get_rect(center=(WIDTH // 2, HEIGHT // 2 +  
y_offset))  
    win.blit(text, rect)
```

```
def show_retry_quit():  
    text = SMALL_FONT.render("Press R to Retry    ESC to Quit",  
True, BLACK)  
    rect = text.get_rect(center=(WIDTH // 2, HEIGHT - 30))  
    win.blit(text, rect)
```

```
def reset_game():  
    global word_to_guess, guessed, hangman_status, game_over,  
message  
    word_to_guess = random.choice(word.words).upper()  
    guessed = []  
    hangman_status = 0  
    game_over = False  
    message = ""
```

```
def redraw_game():  
    win.fill(WHITE)  
    draw_hangman(win, hangman_status)  
    display_word()  
    draw_letters()  
    draw_stats()
```

```

if not game_over:
    footer = SMALL_FONT.render(" ", True, BLACK)
    rect = footer.get_rect(center=(WIDTH // 2, HEIGHT - 30))
    win.blit(footer, rect)

if game_over:
    # Create blurred background
    temp = win.copy()
    w, h = temp.get_size()
    small = pygame.transform.smoothscale(temp, (w//8, h//8))
    blurred = pygame.transform.smoothscale(small, (w, h))
    win.fill(WHITE) # clear
    win.blit(blurred, (0,0))
    # Draw overlays
    color = GREEN if message == "You Win!" else RED
    show_message(message, color, -20)
    show_retry_quit()
pygame.display.update()

```

```

run = True
clock = pygame.time.Clock()
while run:
    clock.tick(60)
    redraw_game()
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            run = False
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                run = False

```

```

if game_over:
    if event.key == pygame.K_r:
        reset_game()
else:
    if event.unicode.isalpha():
        l = event.unicode.upper()
        if l not in guessed:
            guessed.append(l)
            if l not in word_to_guess:
                hangman_status += 1
                if hangman_status >= max_wrong:
                    message = f"You Lose! Word:
{word_to_guess}"

                    game_over = True
            elif all(c in guessed for c in
word_to_guess):

                message = "You Win!"
                game_over = True

pygame.quit()

```

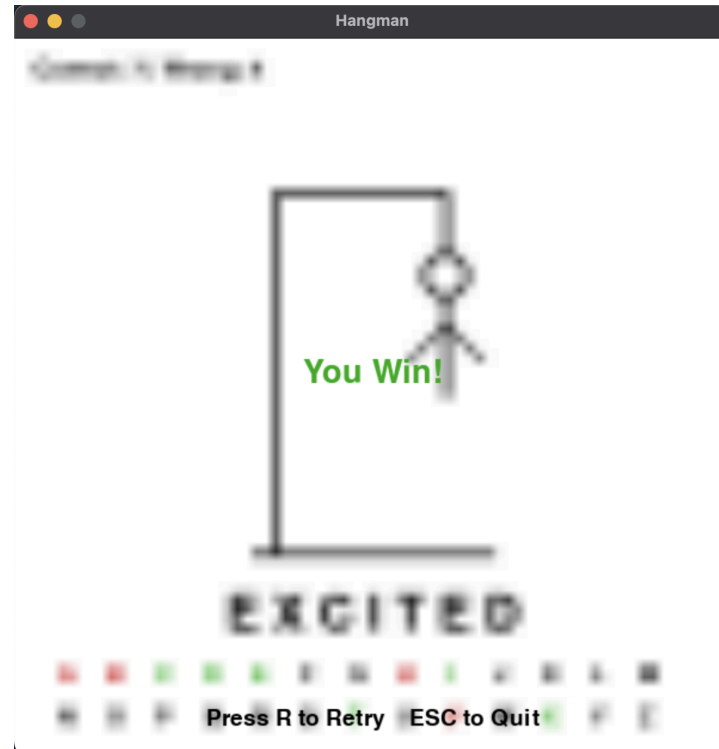
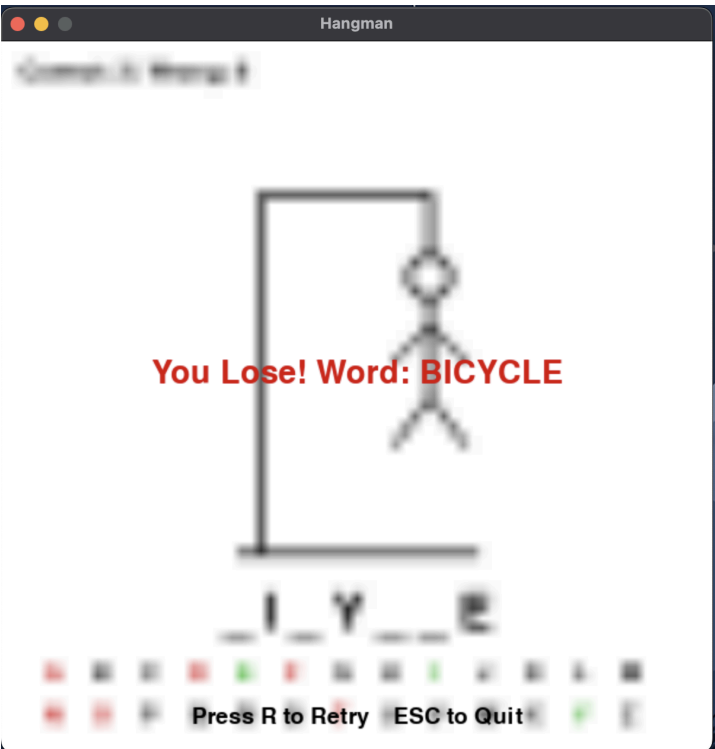
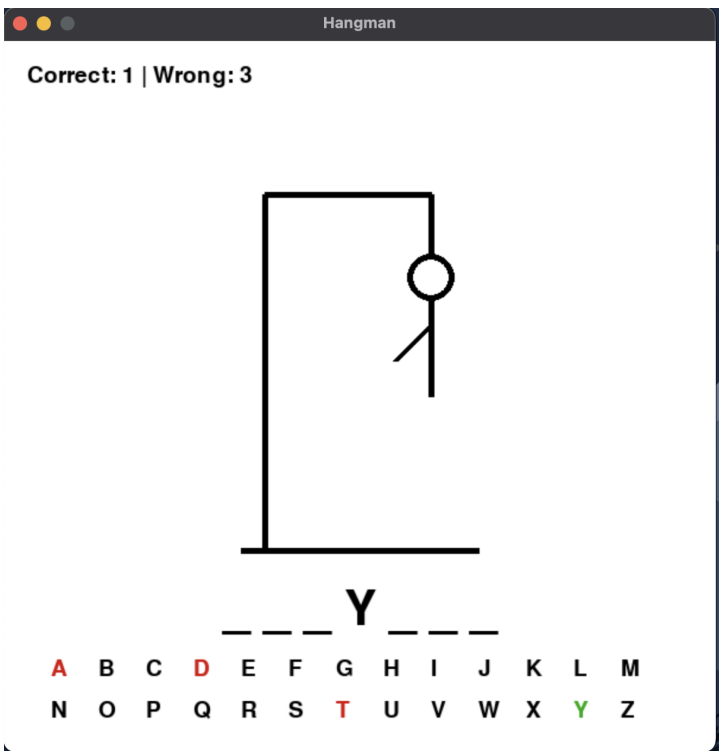
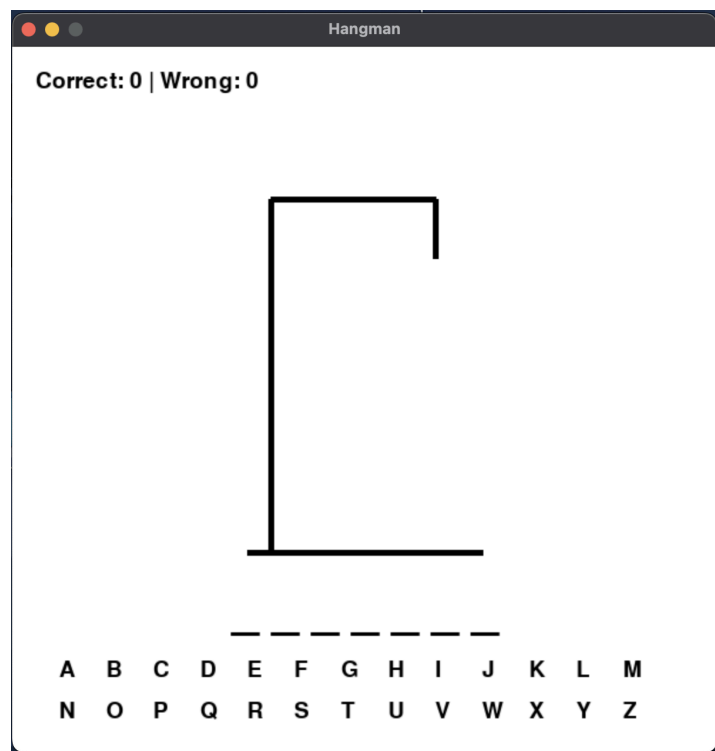
word.py

```
words = [
```

```
    "apple", "banana", "basket", "bottle", "button", "camera",  
    "castle", "cheese", "circle", "cloud", "coffee", "cookie",  
    "cousin", "desert", "doctor", "drawer", "eagle", "engine",  
    "family", "flower", "forest", "friend", "garden", "guitar",  
    "hammer", "holiday", "island", "jacket", "jungle", "kitchen",  
    "ladder", "lemon", "lizard", "mirror", "monkey", "mountain",  
    "napkin", "needle", "notebook", "orange", "pencil", "planet",  
    "pocket", "potato", "rabbit", "rainbow", "remote", "river",  
    "rocket", "school", "season", "shadow", "shower", "silver",  
    "sister", "slipper", "smile", "soccer", "spider", "spring",  
    "square", "stairs", "station", "street", "summer", "sunset",  
    "sweater", "switch", "tablet", "temple", "tiger", "toilet",  
    "tomato", "tooth", "travel", "trophy", "turtle", "vacuum",  
    "valley", "vanish", "village", "violin", "wallet", "window",  
    "winter", "wizard", "worker", "writer", "zebra", "zipper",  
    "anchor", "balloon", "battery", "blanket", "branch",  
    "bridge", "bucket", "candle", "carpet", "cement"
```

```
]
```


PROGRAM OUTPUT



CONCLUSION

The Hangman project successfully recreates the classic word-guessing game in a fun and interactive way using Python and Pygame. It demonstrates how simple programming logic can be combined with graphical elements to create an enjoyable user experience.

Through this project, we learned how to handle graphics, events, and user input in Pygame while applying problem-solving and logical thinking skills. Overall, the game not only provides entertainment but also helps improve vocabulary and spelling in an engaging way.

BIBLIOGRAPHY

Pygame Documentation

<https://www.pygame.org/docs/>

Python Official Documentation

<https://docs.python.org/3/>

TutorialsPoint – *Python Game Development with Pygame*

GeeksforGeeks – *Building Games with Python and Pygame*

Stack Overflow – *Community discussions and troubleshooting for Python and Pygame*